ILLINOIS

Search in course

Search

English

B

Hide menu

- **Week 10 Information**
- **Columnar Storage and the modern Data Warehouse**
- **Data Lake**
- **Spark SQL**
- **Other Cloud Analytics**
- **Week 10 Graded Activities**

Graded Assignment: Week 10 Quiz

. Duration: 1 hour1h

Programming Assignment: Machine Problem 8 : SparkSQL

. Duration: 4 hours4h

Programming Assignment: Machine Problem 9 V3: Analytics - Visualization

. Duration: 6 hours6h

1. CS 498: Cloud Computing Applications
2. Week 10
3. Machine Problem 8 : SparkSQL

PreviousNext

Information:

After the due date, 10% will be deducted from your grade every week until you submit this. Learn more

Programming Assignment: Machine Problem 8 : SparkSQL

You have not submitted.

Deadline

Pass this assignment by Apr 13, 11:59 PM CDT

InstructionsMy submissionsDiscussions

Instructions

1. Overview

Welcome to the SparkSQL Machine Practice. This MP has both java and python templates; please use whichever you prefer. This MP requires Docker Desktop.

2. General Requirements

This assignment will be graded based on **JDK 21** and **Python 3**

Please note that our grader runs on a docker container and is **NOT connected to the internet**. Therefore, **no additional libraries are allowed** for this assignment. Also, you will **NOT be allowed to create any file or folder outside the current folder** (that is, you can only create files and folders in the folder that your solutions are in).

3. Set up the environment

Step 1: Download the docker file, build a docker image and run the container. If you have already created the container, don't create a new one. Start the one you already have, as all your files are already stored there.

11

docker start -a mp8-cntr

4. Procedures

Step 2: Enter the container 'mp8-cntr' after the setup. Clone the repository again inside the container, change the working directory to java/ or python/ , and run:

1

2

3

4

5

6

7

git clone https://github.com/UIUC-CS498-Cloud/MP8_SparkSQL_Template.git

Use Java Templates

cd MP8_SparkSQL_Template/java

Use Python Templates

cd MP8_SparkSQL_Template/python

Step 3: Finish the following exercises by editing the **TODO** sections in the provided templates files. Please note that you are **NOT allowed to import any additional libraries.**

- We will adhere to the best practices for using Spark 3. Be aware that utilizing deprecated APIs could result in an inability to pass the autograder.
- Each exercise should be implemented in one file only. Multiple-file implementation is not allowed.
- The code should be compiled and run on the sample Docker image.

- You should run the parts in the order described below. Otherwise, you might not get the correct results. For example, you should create the RDDs/DataFrames first and then list them.
- Remember that the output is case-sensitive.

Exercise A: Setup

You will create an RDD and a DataFrame object using Spark's API. The template for this exercise is in the following file: **MP8_PartA.java / MP8_PartA.py**.

1. We are targeting at the public gbooks dataset (http://storage.googleapis.com/books/ngrams/books/googlebooks-eng-all-1gram-20120701-a.gz) and provide you with a small subset of this dataset to explore locally. Each of the lines is tab-separated data. As an example,

ATTRIBUTE 1818 11 4

This line tells us that in 1818, the word "ATTRIBUTE" occurred 11 times overall, in 4 distinct books of our sample. We name the columns "word", "year", "frequency", and "books".

2. Write a function to load it in an RDD & DataFrame. Print your DataFrame's schema.

3. The grader is using a larger dataset, so it is a good idea to test your code first on the provided small subset of data before proceeding to test it on the entire dataset.

NOTE: Your programs should always assume the "gbooks" file is in the current directory under java/ or python/.

After downloading and renaming the dataset, the following are the commands we use to run your spark file:

1

2

3

4

5

Python

spark-submit MP8_PartA.py

Java

./run.sh MP8_PartA Output_PartA

Your output should contain the following. Please note that your schema should be aligned with these outputs.

1

2

3

4

5

root

|-- word: string (nullable = true)

|-- year: integer (nullable = true)

|-- frequency: integer (nullable = true)

|-- books: integer (nullable = true)

Exercise B: Counting

How many lines does the RDD contain? Answer this question using the Spark SQL API or dataframe APIs.

The template for this exercise is in the following file: **MP8_PartB.java / MP8_PartB.py**.

The following are the commands we use to run your file:

1

2

3

4

Java

./run.sh MP8_PartB Output_PartB

Python

spark-submit MP8_PartB.py

Your output should contain:

1

2

3

4

5

+--------+

|count(1)|

+--------+

| 50013|

+--------+

Exercise C: Filtering

Count the number of appearances of the word 'ATTRIBUTE'.

The template for this exercise is in the following file: **MP8_PartC.java / MP8_PartC.py**.

The following are the commands we use to run your file:

1

2

3

4

Java

./run.sh MP8_PartC Output_PartC

Python

spark-submit MP8_PartC.py

Your output should contain:

1

2

3

4

5

+--------+

|count(1)|

+--------+

| 11|

+--------+

Exercise D: MapReduce

List the 3 most frequent words with the counts of their appearances, **final answer should be sorted first by descending order of counts, then by descending order of words.** This sorting order must be followed to avoid arbitrary sorted answers due to ties in word counts.

The template for this exercise is in the following file: **MP8_PartD.java / MP8_PartD.py**.

The following are the commands we use to run your file:

1

2

3

4

Java

./run.sh MP8_PartD Output_PartD

Python

spark-submit MP8_PartD.py

Your output should look like the following:

1

2

3

4

5

6

7

8

```
+-------------+--------+
|       word|count(1)|
+-------------+--------+
|   ATTRIBUTE|     11|
|approximation|      4|
|   agast_ADV|      4|
+-------------+--------+
only showing top 3 rows
```

Note that we expect to see the last line "only showing top 3 rows" in your output. To do that, use the **show(n)** function for the SparkSQL query. Also note that your top 3 output may not look exactly like the above when run locally as there are ties present among some words.

Exercise E: Joining

The following code creates a new DataFrame out of the 'df' with a much smaller size.

1

2

```
df2 = df.select("word", "year").distinct().orderBy("year", "word").limit(100)

df2.createOrReplaceTempView('gbooks2') # Register table name for SQL
```

Now we will perform a JOIN operation on the 'df2'. Please find the number of pairs of words in 'df2' that appeared in the same year. **Answer this question using purely the Spark SQL API.**

Hint: Do a self-join on 'df2' in lines with the same 'year' values

In case you haven't used a self-join before, you may want to break down the steps as the following:

1. retrieve all records from the cross join of the "gbooks2" table with itself

2. impose the condition where the "year" field of the first instance of the gbooks table is equal to the "year" field of the second instance of the gbooks table

The template for this exercise is in the following file: **MP8_PartE.java / MP8_PartE.py**.

The following are the commands we use to run your file:

1

2

3

4

Java

./run.sh MP8_PartE Output_PartE

Python

spark-submit MP8_PartE.py

The output should look similar to below:

1

310

Exercise F: Frequency Increase

Now we want to analyze the frequency increase of words starting from the year **1500** to the year **2000**. For example, a word example1 occurs multiple times across different years:

1

2

3

4

5

6

example1 1982 2 1

example1 1900 3 1

example1 1800 2 1

example1 1995 2 1

example1 1980 5 1

example1 1985 2 1

The word example1 occurs 2 times in the year 1800 and 16 times at the end of year 2000. Thus there is an increase of 14. The output for this should be like this:

1

2

3

4

5

+---------+--------------+

| word|total_increase|

+---------+--------------+

| example1| 14|

+---------+--------------+

To clarify, we want to analyze the increase in word frequency between the first occurrence of a word after 1500 and the last occurrence before the end of 2000 (inclusive of 2000). For simplicity, we consider the first occurrence of the word as the starting point to count frequency increases. For instance, in the example below, we would treat the entry **example1 1800 2 1** as the starting point:

Hint: You can first create a window for each word, sorted by year, and use the lag function to calculate the frequency increase for each year and then sum up the increases. Specifically, for the example above, you can add an additional column named frequency_increase, as shown below. To obtain the total_increase for each word, sum up the frequency_increase values and rename the resulting column as total_increase.

1

2

3

4

5

6

7

8

9

10

```
+---------+----+---------+-----+-----------------+
|    word|year|frequency|books|frequency_increase|
+---------+----+---------+-----+-----------------+
| example1|1800|        2|    1|                3|
| example1|1900|        3|    1|                5|
| example1|1980|        5|    1|                2|
| example1|1982|        2|    1|                2|
| example1|1985|        2|    1|                2|
| example1|1995|        2|    1|                0|
+---------+----+---------+-----+-----------------+
```

For Python users, please refer to the below documentation for lag and window

For Java users, please refer to this link: lag and window

The template for this exercise is in the following file: **MP8_PartF.java / MP8_PartF.py**.

The following are the commands we use to run your file:

1

2

3

4

Java

./run.sh MP8_PartF Output_PartF

Python

spark-submit MP8_PartF.py

The top 5 rows should be similar to the output, and **the grader is looking for the top 20 rows sorted by the total_increase**:

1

2

3

4

5

6

7

8

9

10

```
+--------------+--------------+
|          word|total_increase|
+--------------+--------------+
|  another_DET|      1176086|
|     actually|       366565|
|   abroad_ADV|       158396|
|     assembly|       110757|
|appointed_VERB|       102652|
+--------------+--------------+
only showing top 5 rows
```

How to Submit:

For this MP, please compress all of your source files as required below, zip it, and submit the zip file.

Java

After finishing the assignment, compress the source files in **src/** together with the **pom.xml** file in their working directory into a zip file. Please do not include the parent folder, or the gbooks file. To create a submission file, you can follow the script below in your terminal:

1

2

For Java Submission

$ zip -r mp8java.zip src/ pom.xml

Python

After finishing the assignment, compress all six files (MP8_PartX.py) into a zip file. Make sure you only compress MP8_PartX.py into the zip file, not the parent_folder. You submission should be a zip file including five ./MP8_PartX.py. To create a submission file, you can follow the script below in your terminal:

1

2

For Python Submission, run under python/ folder

$ zip -r mp8py.zip *.py

The autograder will run your code using the command lines in instructions and compares the files produced by your code files to the expected outputs.

Note: For all the parts A, B, C, D, E and F, you may not get points if your code fails to produce the **same results** as in the instructions. Note that we use a different gbooks file to grade your submission.

How to submit

When you're ready to submit, you can upload files for each part of the assignment on the "My submissions" tab.

Like

Dislike

Report an issue